

Riassunto del workflow Python e SQL

1. Fase Python: caricamento del dataset

La prima fase del progetto è stata svolta in Python utilizzando la libreria pandas.

Il dataset originale DataCo Supply Chain è stato importato da file CSV. Durante il caricamento è emerso un problema di encoding, quindi il file è stato letto utilizzando l'encoding latin1.

Dopo il caricamento, è stata eseguita una prima esplorazione del dataset per analizzare:

- numero di righe e colonne;
- nomi delle colonne;
- tipi di dato;
- valori mancanti;
- righe duplicate;
- unicità degli identificativi principali.

È stata poi creata una copia di lavoro del dataset, in modo da preservare il file originale.

2. Fase Python: rinomina e riorganizzazione delle colonne

Le colonne originali sono state rinominate in formato snake_case, così da renderle più leggibili e più semplici da utilizzare in SQL e Power BI.

Le variabili sono state riorganizzate in gruppi logici:

- informazioni cliente;
- informazioni ordine;
- località di consegna;
- località di registrazione/acquisto;
- prodotto e categoria;
- dettagli della riga d'ordine;
- vendite e profitti;
- spedizione e consegna.

Una decisione importante è stata chiarire il significato dei campi geografici. Le colonne relative a Customer City, Customer State, Customer Country, ecc. sono state interpretate come informazioni legate al punto di registrazione/acquisto, non come indirizzo personale del cliente.

Per questo motivo sono state rinominate come:

- store_city;
- store_state;
- store_country;
- store_zipcode;
- store_latitude;
- store_longitude.

Le colonne relative all'ordine sono invece state interpretate come informazioni sulla destinazione di consegna:

- delivery_city;
- delivery_state;
- delivery_region;
- delivery_country;
- delivery_market.

3. Fase Python: eliminazione delle colonne non utili

Alcune colonne sono state eliminate perché non utili all'analisi, ridondanti oppure non adatte al modello finale.

Sono state rimosse colonne come:

- email cliente;
- password cliente;
- link prodotto;
- descrizione prodotto;
- indirizzo stradale dello store;
- zipcode della consegna;
- stato stock del prodotto.

Questa fase ha permesso di semplificare il dataset e ridurre le informazioni non necessarie prima dell'importazione in SQL.

4. Fase Python: correzione dei tipi di dato

Sono stati corretti alcuni tipi di dato.

Le colonne data sono state convertite in formato datetime:

- order_date;
- shipping_date.

La colonna store_zipcode è stata gestita in modo da mantenere un formato coerente e compatibile con l'esportazione finale.

Sono stati poi eseguiti controlli su:

- valori mancanti;
- duplicati;
- unicità di order_item_id;
- coerenza tra colonne chiave.

L'analisi ha confermato che il dataset ha granularità a livello di riga d'ordine, cioè: una riga rappresenta un singolo order item.

Questo è stato verificato perché order_item_id è univoco e il numero di valori distinti coincide con il numero totale di righe.

5. Fase Python: validazione delle relazioni tra colonne

Durante la fase di controllo sono state confrontate alcune colonne chiave per capire come costruire il modello relazionale in SQL.

È stato verificato che:

- order_item_product_card_id corrisponde a product_card_id;
- order_customer_id corrisponde a customer_id;
- product_category_id corrisponde a category_id.

Questi controlli hanno mostrato che alcune colonne sono ridondanti, ma utili per costruire le relazioni tra fact table e dimension table.

In particolare, product_category_id è stato mantenuto nella tabella prodotto come chiave esterna verso dim_category.category_id.

6. Fase Python: correzione di anomalie nei dati

Sono state individuate alcune anomalie nei campi relativi alla località dello store.

Alcune righe presentavano valori mancanti o incoerenti in campi come città, stato e zipcode. Queste righe sono state corrette manualmente utilizzando combinazioni valide già presenti nel dataset.

Tra le correzioni effettuate:

- Elk Grove, CA, zipcode 95758;
- El Monte, CA, zipcode 91732.

Dopo queste correzioni, i valori mancanti relativi a store_zipcode sono stati risolti.

7. Fase Python: standardizzazione testuale ed esportazione finale

Le colonne testuali sono state pulite eliminando eventuali spazi iniziali e finali.

È stata inoltre standardizzata la dicitura del paese:

- EE. UU. è stato sostituito con USA.

Al termine della pulizia, il dataset finale è stato esportato in formato CSV con il nome: dataco_cleaned_final.csv

Questo file è stato poi utilizzato come base per l'importazione in MySQL.

Workflow SQL

8. Fase SQL: creazione del database e della staging table

Dopo la fase di pulizia in Python, il dataset finale `dataco_cleaned_final.csv` è stato importato in MySQL per costruire un modello dati più strutturato e adatto al collegamento con Power BI.

È stato creato un database dedicato al progetto DataCo Supply Chain ed è stata creata una staging table chiamata `dataco_cleaned_final_2`, utilizzata come tabella temporanea/intermedia per caricare il CSV pulito esportato da Python.

La staging contiene tutte le informazioni necessarie per la modellazione successiva:

- dati cliente;
- dati ordine;
- dati prodotto;
- dati categoria;
- dati store;
- dati località di consegna;
- metriche di vendita e profitto;
- dati di spedizione e consegna e ritardo.

La staging table rappresenta quindi il punto di partenza per costruire le tabelle finali del modello.

9. Fase SQL: importazione del CSV pulito

Il file `dataco_cleaned_final.csv` è stato importato in MySQL tramite il comando `LOAD DATA INFILE`.

L'importazione è stata eseguita usando:

- character set `utf8mb4`;
- campi separati da virgola;
- valori testuali racchiusi tra doppi apici;
- esclusione della prima riga contenente gli header.

Dopo l'importazione è stato eseguito un controllo sul numero totale di righe e sul numero di `order_item_id` distinti.

Il risultato è stato:

- righe totali: 180519;
- `order_item_id` distinti: 180519.

Questo conferma che l'importazione è avvenuta correttamente e che il dataset mantiene la granularità corretta.

10. Fase SQL: progettazione del modello logico

Dalla staging table è stato costruito un modello analitico composto da tabelle dimensionali e una fact table centrale.

Il modello finale è composto da:

- `dim_category`;
- `dim_product`;
- `dim_customer`;
- `dim_store`;
- `dim_delivery_location`;
- `dim_order`;
- `fact_order_items`.

La tabella centrale è `fact_order_items`, perché il livello di dettaglio principale del dataset è la riga d'ordine.

11. Fase SQL: creazione delle dimension table

Le dimension table sono state create a partire dai valori distinti presenti nella staging table.

- **dim_category** contiene le categorie prodotto, identificate da `category_id`.
- **dim_product** contiene le informazioni sui prodotti, con `product_card_id` come chiave primaria e `product_category_id` come chiave esterna verso `dim_category`.
- **dim_customer** contiene le informazioni cliente, con `customer_id` come chiave primaria;
- **dim_store** contiene le informazioni relative allo store / punto di registrazione o acquisto. Poiché `store_department_id` non era univoco, è stata creata una chiave tecnica `store_id`.
- **dim_delivery_location** contiene le informazioni relative alla località di consegna. Anche in questo caso, non essendoci una chiave naturale univoca, è stata creata una chiave tecnica `delivery_location_id`.

- **dim_order** contiene le informazioni a livello ordine, tra cui order_id, order_date, transaction_type, order_status.

La fact table fact_order_items contiene:

- la chiave primaria order_item_id;
- le chiavi esterne verso le dimensioni;
- quantità ordinate;
- prezzo prodotto;
- sconti;
- totale riga ordine;
- vendite;
- profitto;
- modalità di spedizione;
- giorni di spedizione programmati;
- giorni di spedizione effettivi;
- stato consegna;
- rischio di consegna in ritardo.

Per popolare fact_order_items, sono state utilizzate INNER JOIN con dim_store e dim_delivery_location, necessarie per recuperare le chiavi tecniche generate nelle dimensioni.

Le join sono state costruite confrontando i campi descrittivi che identificano in modo univoco store e località di consegna.

13. Fase SQL: controlli finali e creazione viste

Dopo la creazione del modello, sono stati eseguiti controlli finali sul numero di righe di tutte le tabelle.

Il controllo principale ha verificato che fact_order_items contenesse ancora **180.519 righe** e **180.519 order_item_id** distinti.

Questo ha confermato che nessuna riga era stata persa durante la trasformazione dal dataset flat al modello relazionale.

In una seconda fase SQL sono state create alcune **view per Power BI**, con l'obiettivo di semplificare il collegamento e rendere più chiari i campi da usare nel report.

La vista principale vw_fact_order_items arricchisce la fact table con informazioni provenienti da dim_order, come data ordine, tipo transazione e stato ordine.

Nella vista fact sono stati creati anche campi utili per il reporting, come:

- order_date_only;
- shipping_delay_days;
- late_delivery_risk_label.

Sono state create anche viste descrittive per prodotto e cliente, così da rendere più semplice l'utilizzo dei dati in Power BI.

Il risultato finale della fase SQL è un modello strutturato e più adatto all'analisi BI, collegabile a Power BI tramite viste dedicate.

Workflow Power BI

14. Fase Power BI: creazione del database e della staging table

Il modello creato in SQL è stato collegato a Power BI per realizzare la dashboard finale del progetto.

In Power BI sono state importate le viste e le tabelle necessarie per costruire il report:

- dim_product (vista);
- dim_customer (vista);
- dim_store;
- dim_delivery_location;
- fact_order_items (vista).

Il modello Power BI mantiene la logica costruita in SQL, con una fact table centrale a livello di order_item_id e diverse dimensioni collegate.

È stata creata una tabella calendario per supportare le analisi temporali e il filtro per anno.

Sono state create misure DAX per calcolare i principali indicatori del report, tra cui:

- vendite totali;
- profitto totale;
- margine di profitto;
- ordini totali;
- clienti totali;
- average order value;
- spesa media per cliente;
- ordini medi per cliente;
- late delivery rate;
- on-time rate;
- giorni medi di spedizione;
- giorni medi di ritardo.

Una scelta importante ha riguardato la granularità delle metriche. Il dataset non è a livello di singolo ordine, ma a livello di order item: ogni riga rappresenta un prodotto contenuto in un ordine.

Questo significa che uno stesso order_id può comparire più volte, quando un ordine contiene più prodotti. Infatti, nel dataset order_item_id è univoco, mentre order_id può ripetersi.

Questa distinzione è stata necessaria perché il dataset ha granularità a livello di riga ordine, ma alcune KPI devono essere lette a livello ordine.

La dashboard finale è stata organizzata in tre pagine principali:

- Overview mostra una sintesi generale delle performance:
 - vendite totali;
 - profitto totale;
 - numero ordini;
 - margine di profitto;
 - late delivery rate;
 - ritardo medio;
 - andamento annuale di vendite e margine;
 - categorie principali per vendite e relativo profitto;
 - distribuzione geografica dello store profit.
- Delivery approfondisce le performance di spedizione e consegna:
 - late delivery rate;
 - on-time rate;
 - ordini in ritardo;
 - giorni medi di spedizione;
 - giorni medi di ritardo;
 - ordini totali;
 - performance per shipping mode;
 - late delivery rate per paese;
 - confronto tra giorni di spedizione effettivi e programmati;
 - distribuzione degli stati di consegna.

Nella pagina Delivery è stata applicata una formattazione condizionale al Late Delivery Rate: quando il valore supera la soglia del 20%, viene evidenziato graficamente per segnalare una criticità.

La soglia del 20% è stata usata come benchmark visivo di warning, utile anche per riconoscere più facilmente eventuali scenari meno critici quando vengono applicati filtri.

- Customer & Store analizza il comportamento commerciale per cliente e punto vendita:
 - clienti totali;
 - spesa media;
 - ordini medi;
 - average order value;
 - vendite totali;
 - margine di profitto;
 - performance delle store cities;
 - vendite per metodo di pagamento;
 - vendite per segmento cliente.

Per migliorare la lettura geografica della mappa, è stato creato un campo concatenato di localizzazione store, combinando città, stato e paese.

È stata inoltre creata una classificazione per distinguere le località di Puerto Rico da quelle degli USA, usata come legenda nella mappa.

Il report include slicer per filtrare i dati per anno e per area geografica.

È stata inserita una nota tecnica per segnalare che il 2018 contiene dati parziali e deve quindi essere interpretato con cautela nei confronti degli anni completi.

È stata aggiunta una pagina nascosta di Technical Notes, utile per documentare:

- obiettivo del report;
- struttura delle pagine;
- modello dati;
- granularità del dataset;
- uso di misure DAX e colonne calcolate;
- logica del Late Delivery Rate;
- nota sui dati parziali del 2018.

Il risultato finale è una dashboard Power BI interattiva, pensata per analizzare vendite, profitti, clienti, store, spedizioni e rischio di consegna in ritardo, mantenendo coerenza tra granularità del dataset, modello SQL e misure DAX.